

Deep Network Structures and Parameters for Speaker Identification

Architecture, Hyperparameter, and Deployment Reference

arXiv-style technical report

Scope. This report concentrates on the internal network structures and tunable parameters used to build modern speaker-identification systems. It distinguishes paper-canonical configurations from toolkit variants, because a model name such as “ECAPA-TDNN” or “ResNet34” does not uniquely determine channel widths, pooling, embedding size, or classifier loss. The report covers x-vector TDNNs, ECAPA-TDNN, ResNet/Res2Net/ERes2Net, TitaNet, CAM++, MFA-Conformer, WavLM/UniSpeech-SAT-style self-supervised encoders, and Audio Spectrogram Transformer (AST/AST-Tiny) adaptations. The intended input regime includes 16-kHz speech and both closed-set and open-set speaker identification.

August 2026

Technical synthesis based on primary papers and current open-source reference configurations

Abstract

A speaker-identification network must transform variable-duration speech into a representation that is highly sensitive to speaker identity but comparatively insensitive to phonetic content, channel, noise, and session. Modern systems do this with an encoder, a temporal aggregation layer, an embedding projection, and a discriminative training head. This report gives a detailed, implementation-oriented description of the principal deep-network families used for speaker identification and verification, with explicit layer dimensions, kernel sizes, dilation factors, attention parameters, embedding sizes, training losses, and front-end settings. Canonical x-vector and ECAPA-TDNN networks are reconstructed from the original papers; TitaNet parameters are cross-checked against current NVIDIA NeMo configurations; CAM++ parameters are tied to the 3D-Speaker/FunASR implementation; MFA-Conformer values are taken from the original paper; ERes2NetV2 values are taken from current 3D-Speaker recipes; WavLM Base/Large configurations are cross-checked against Microsoft/Hugging Face model metadata; and AST-Tiny values are tied to the distilled DeiT-Tiny backbone used in the original AST code path. The report then explains how each parameter changes receptive field, parameter count, compute, embedding quality, robustness, and open-set behavior. It closes with three reproducible reference configurations for edge, balanced, and high-accuracy systems, plus an AST-Tiny transfer-learning configuration for 16-kHz speaker identification.

Keywords: speaker identification, speaker verification, speaker embedding, x-vector, ECAPA-TDNN, ResNet, Res2Net, TitaNet, CAM++, Conformer, WavLM, AST, transfer learning.

Contents

1	Speaker identification as a deep embedding problem	4
2	Parameter taxonomy: what must be frozen for reproducibility	4
3	Acoustic front-end parameters	5
3.1	16-kHz reference front end	5
3.2	Time-frequency representation	5
3.3	Training crop duration	6
4	Canonical x-vector TDNN	6
4.1	Parameters that matter most	6
5	ECAPA-TDNN: canonical structure and parameters	7
5.1	Canonical high-capacity ECAPA configuration	7
5.2	SE-Res2 parameters	7
5.3	Channel-dependent attentive statistics pooling	8
6	ResNet, Res2Net, and ERes2Net families	8
6.1	Representative speaker ResNet34-SE	8
6.2	Res2Net and ERes2Net	8
6.3	Meaning of Res2Net parameters	9

7 TitaNet: depthwise-separable speaker encoder	9
7.1 TitaNet-L current reference configuration	9
7.2 TitaNet-S current reference configuration	10
7.3 Depthwise-separable parameter reduction	10
8 CAM++ / CAMPPlus	10
8.1 Current CAMPPlus parameters	10
8.2 Why the dense growth rate matters	11
9 MFA-Conformer	11
9.1 Paper configuration	11
9.2 Conformer block equations	11
10 Self-supervised Transformer encoders: WavLM and UniSpeech-SAT	12
10.1 WavLM Base / Base+	12
10.2 WavLM Large	12
10.3 UniSpeech-SAT	13
11 Audio Spectrogram Transformer and AST-Tiny for SID	13
11.1 AST-Tiny speaker-identification structure	13
11.2 Pretraining interpretation	14
12 Pooling structures and parameters	14
12.1 Mean and statistics pooling	14
12.2 Attentive statistics pooling	14
12.3 Multi-layer aggregation	14
12.4 CLS-token pooling	14
13 Speaker embedding dimension	14
14 Loss-head parameters	15
14.1 Cross entropy	15
14.2 AAM-Softmax	15
14.3 Classifier parameter count	15
15 Training and optimization parameters	15
16 Closed-set versus open-set network-head design	16
16.1 Closed-set classifier	16
16.2 Open-set embedding system	16

17 Architecture comparison	16
18 Parameter-sensitivity analysis	17
18.1 Channels versus depth	17
18.2 Kernel size versus dilation	17
18.3 Attention heads	18
18.4 Embedding dimension versus classifier size	18
19 Transfer learning parameter strategy	18
19.1 General two-stage recipe	18
19.2 AST-Tiny-specific starting values for 16-kHz SID	19
20 Three engineering reference configurations	19
20.1 Edge / low-compute system	19
20.2 Balanced general-purpose system	20
20.3 High-accuracy server system	20
21 Implementation checklist	20
22 Conclusions	21
A Appendix A: concise formulas for parameter budgets	21
A.1 1D convolution	21
A.2 Depthwise-separable 1D convolution	21
A.3 Transformer self-attention projections	21
B Appendix B: recommended open-set backend parameters	22

1 Speaker identification as a deep embedding problem

Speaker identification (SID) asks which person produced an utterance. A closed-set system assumes the speaker is one of N registered speakers; an open-set system must additionally reject unregistered speakers. Deep networks are most reusable when they learn an embedding extractor

$$f_{\theta} : \mathbf{X} \in \mathbb{R}^{T \times F} \rightarrow \mathbf{z} \in \mathbb{R}^d, \quad (1)$$

where $\mathbf{X}$ is a time-frequency feature sequence, T is the frame count, F the acoustic feature dimension, and d the speaker-embedding dimension.

For dynamic enrollment, each enrolled speaker i is represented by a normalized prototype $\mathbf{g}_i$, and cosine scoring is

$$s_i = \frac{\mathbf{z}^{\top} \mathbf{g}_i}{\|\mathbf{z}\|_2 \|\mathbf{g}_i\|_2}. \quad (2)$$

Closed-set SID chooses $\arg \max_i s_i$. Open-set SID adds threshold τ :

$$\hat{y} = \begin{cases} \arg \max_i s_i, & \max_i s_i \geq \tau, \\ \text{unknown}, & \max_i s_i < \tau. \end{cases} \quad (3)$$

Thus the deep-network design objective is not merely classification accuracy on training identities. It is the geometry, invariance, and calibration quality of the embedding space.

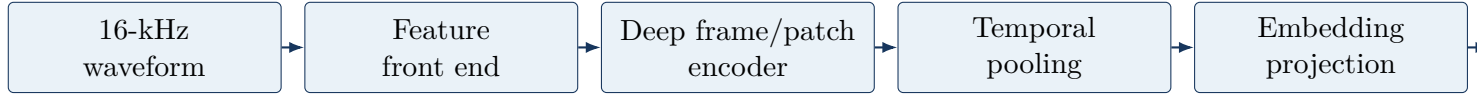


Figure 1: Reusable embedding-centered speaker-identification architecture. The training classification head is usually removed at deployment.

2 Parameter taxonomy: what must be frozen for reproducibility

A model name alone is not enough. A reproducible speaker network specification must freeze the parameters in Table 1.

Table 1: Speaker-network parameter taxonomy.

Parameter class	Examples	Why it matters
Waveform	sample rate, mono/stereo, normalization	Controls available bandwidth and compatibility with pretrained front ends.
Spectral front end	Mel bins, FFT, window, hop, pre-emphasis, mean normalization	Defines time/frequency resolution and input dimension.
Temporal encoder	channels, layers, kernel, dilation, stride, residual topology	Sets receptive field, compute, capacity, and robustness to duration.
Multi-scale structure	Res2Net scale, DenseNet growth, branch widths	Controls simultaneous short/long context modeling.
Attention	heads, key/query dimension, SE bottleneck, context mask	Controls channel/frame selectivity and compute.

Parameter class	Examples	Why it matters
Pooling	mean/std, attentive stats, multi-granularity, CLS token	Determines how variable-length evidence becomes one utterance vector.
Embedding	dimension d , normalization, projection depth	Changes storage/search cost and class-separation capacity.
Training head	number of classes, AM/AAM margin, scale, GE2E/triplet params	Determines discriminative geometry during training.
Optimization	optimizer, LR, weight decay, batch structure, crop length	Can change results as much as architecture.
Open-set backend	enrollment count, prototype rule, threshold, cohort, ANN top- K	Determines real deployed identification behavior.

3 Acoustic front-end parameters

3.1 16-kHz reference front end

For most modern speech-based SID systems, 16 kHz mono audio is a robust default. A commonly reproducible front end is:

- sample rate $f_s = 16,000$ Hz;
- window 25 ms = 400 samples;
- hop 10 ms = 160 samples;
- FFT size 512;
- 80 Mel filter-bank bins;
- log compression and per-feature or utterance mean normalization.

TitaNet’s current NeMo configuration uses exactly 16 kHz, 25-ms Hann windows, 10-ms stride, $n_{\text{FFT}} = 512$, and 80 Mel bins [8]. MFA-Conformer uses 80-dimensional FBANK, 25-ms windows, and 10-ms frame shift [14]. Current 3D-Speaker CAM++ and ERes2Net recipes also use 80-dimensional FBANK at 16 kHz [11, 5].

3.2 Time-frequency representation

With waveform $x[n]$ and analysis window $w[n]$, the STFT is

$$X(t, k) = \sum_n x[n]w[n - tH]e^{-j2\pi kn/N}. \quad (4)$$

A log-Mel coefficient is

$$M(t, m) = \log \left(\epsilon + \sum_k B_m(k) |X(t, k)|^2 \right). \quad (5)$$

The feature dimension F is normally 64–128; 80 is a widely used speaker-recognition default. More Mel bins preserve spectral detail but increase first-layer compute and may overfit narrow datasets.

3.3 Training crop duration

Crop duration is a network parameter in practice because it defines the temporal evidence seen during optimization. Common values are 2–4 s. The original x-vector paper used chunks averaging about 3 s [1]. MFA-Conformer uses fixed 3-s segments [14]; ERes2NetV2’s current recipe uses 3.0 s [5]. Longer crops can improve stable statistics but reduce batch diversity and increase attention cost.

4 Canonical x-vector TDNN

The 2018 x-vector architecture is still the clearest reference for how deep speaker embeddings are formed [1]. Its original input was 24-dimensional filterbanks, but the topology generalizes directly to 80-dimensional FBANK.

Table 2: Canonical x-vector architecture from Snyder et al. The softmax dimension N is the number of training speakers.

Layer	Temporal context	Total context	Input $\times$ output
frame1	$[t - 2, t + 2]$	5	120×512
frame2	$\{t - 2, t, t + 2\}$	9	1536×512
frame3	$\{t - 3, t, t + 3\}$	15	1536×512
frame4	$\{t\}$	15	512×512
frame5	$\{t\}$	15	512×1500
Statistics pooling	full utterance	T	$1500T \rightarrow 3000$
segment6	utterance	T	3000×512
segment7	utterance	T	512×512
softmax	utterance	T	$512 \times N$

The pooling concatenates mean and standard deviation,

$$\boldsymbol{\mu} = \frac{1}{T} \sum_t \mathbf{h}_t, \quad (6)$$

$$\boldsymbol{\sigma} = \sqrt{\frac{1}{T} \sum_t \mathbf{h}_t \odot \mathbf{h}_t - \boldsymbol{\mu} \odot \boldsymbol{\mu}}, \quad (7)$$

therefore the 1500-channel frame representation becomes a 3000-dimensional utterance representation. The original speaker embedding is taken from segment6 before its nonlinearity. Excluding later training-only layers, the paper reports about 4.2M parameters [1].

4.1 Parameters that matter most

TDNN channels	512 is the main frame width; increasing it adds quadratic-like cost to dense temporal layers.
Frame5 width	1500 makes the statistics vector rich, but doubles to 3000 after mean/std concatenation.
Embedding	512 is larger than many modern 192-D systems; reducing to 192 lowers gallery memory and cosine-search cost.

Context The first three layers create 15-frame effective context before utterance pooling. This is efficient but weaker than modern multi-scale/global encoders.

5 ECAPA-TDNN: canonical structure and parameters

ECAPA-TDNN improves the x-vector by using SE-Res2 blocks, hierarchical feature aggregation, and channel-dependent attentive statistics pooling [2].

5.1 Canonical high-capacity ECAPA configuration

The original study evaluated convolutional widths $C = 512$ and $C = 1024$. For the high-capacity setting:

- input: typically 80 log-Mel/FBANK in modern implementations;
- initial 1D convolution: kernel 5, $C = 1024$;
- three SE-Res2 blocks: $C = 1024$, kernel 3, dilations typically 2, 3, 4;
- Res2Net scale $s = 8$;
- SE and attention bottleneck dimension 128;
- multi-layer aggregation: concatenate block outputs, project to 1536 channels;
- attentive statistics pooling: 1536 channels $\rightarrow$ weighted mean + weighted std = 3072;
- final speaker embedding: 192 dimensions;
- training head: normally AM/AAM-Softmax with one class per training speaker.

The original paper explicitly states $C = 512/1024$, bottleneck 128, Res2 scale 8, and 192 nodes in the final fully connected layer [2]. Current independent ECAPA implementations may report parameter counts from roughly 14.7M to above 20M because the exact channels, head, and auxiliary layers vary; therefore parameter count should always be tied to a concrete implementation, not only the model name.

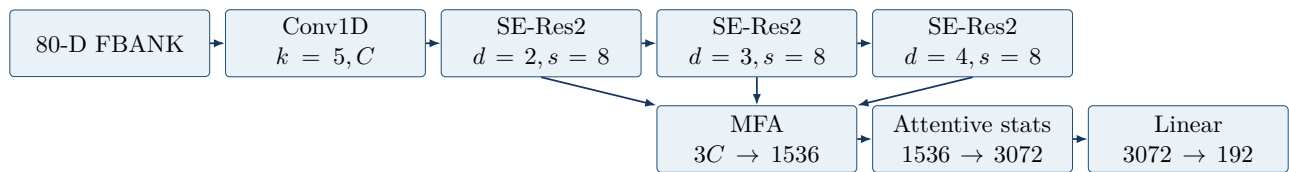


Figure 2: Representative ECAPA-TDNN data path. C is commonly 512 or 1024; the original large system uses $C = 1024$.

5.2 SE-Res2 parameters

A Res2 block divides channels into s groups. With $C = 1024$ and $s = 8$, each group has approximately 128 channels before implementation-specific projections. Larger s creates more internal receptive-field scales without requiring a fully independent branch for every scale. The SE bottleneck $R = 128$ controls the capacity of global channel gating.

5.3 Channel-dependent attentive statistics pooling

For channel c and frame t ,

$$e_{t,c} = \mathbf{v}_c^T f(\mathbf{W}\mathbf{h}_t + \mathbf{b}) + k_c, \quad (8)$$

$$\alpha_{t,c} = \frac{e^{e_{t,c}}}{\sum_{\tau} e^{e_{\tau,c}}}, \quad (9)$$

$$\tilde{\mu}_c = \sum_t \alpha_{t,c} h_{t,c}, \quad (10)$$

$$\tilde{\sigma}_c = \sqrt{\sum_t \alpha_{t,c} h_{t,c}^2 - \tilde{\mu}_c^2}. \quad (11)$$

Unlike ordinary attention with one weight per frame, different channels may emphasize different phonetic regions.

6 ResNet, Res2Net, and ERes2Net families

6.1 Representative speaker ResNet34-SE

Speaker ResNets use 2D convolution on time-frequency maps. A common ResNet34-SE reference has four stages with residual block counts $[3, 4, 6, 3]$ and channels $[32, 64, 128, 256]$, preceded by a 3×3 or related stem. The 2022 FFSVC reference configuration reports a 32-channel stem followed by SE residual stages at 32, 64, 128, and 256 channels [3]. Speaker variants differ in stride placement, pooling, and embedding size; 192 or 256 dimensions are common.

For a residual block,

$$\mathbf{Y} = \mathbf{X} + F(\mathbf{X}; \theta), \quad (12)$$

and an SE gate scales channels using a global descriptor. ResNet advantages are regular 2D computation and strong scaling; disadvantages are frequency-downsampling decisions and potentially high FLOPs.

6.2 Res2Net and ERes2Net

Res2Net introduces internal channel splits to realize multiple receptive-field scales in one block. ERes2Net further fuses local features within a block and global features across scales with attention [4]. Current 3D-Speaker ERes2NetV2 training configuration uses:

- 16 kHz audio, 80-D FBANK;
- 3.0-s training waveform crops;
- $m_channels = 64$;
- embedding 192;
- baseWidth 26;
- scale 2;
- expansion 2;
- 5994 VoxCeleb2 training classes in the referenced recipe;
- 17.8M parameters for the reported ERes2NetV2 model [5, 6].

6.3 Meaning of Res2Net parameters

baseWidth	Controls branch width before scaling; larger values increase per-branch capacity.
scale	Number of internal feature groups/scales. Higher scale adds multi-scale resolution but increases coordination overhead.
expansion	Residual output expansion factor, analogous to bottleneck ResNet expansion.
m_channels	Global width multiplier/stem width; often the dominant parameter-count lever.

7 TitaNet: depthwise-separable speaker encoder

TitaNet is explicitly designed for scalable speaker embeddings using 1D depthwise-separable convolutions, SE context, and attention pooling [7]. Current NVIDIA NeMo configs provide unusually complete reproducibility data [8].

7.1 TitaNet-L current reference configuration

Table 3: Key TitaNet-L parameters in the current NeMo reference config.

Parameter	Value
Sample rate	16 kHz
Mel bins / FFT	80 / 512
Window / hop	25 ms / 10 ms
Main filters	1024
Repeat per central block	3
Dropout	0.1
Depthwise separable	yes
SE context	global (−1)
Block kernels	3, 7, 11, 15, then 1
Final encoder channels	3072
Pooling	attention
Embedding	192
Angular loss scale / margin	30 / 0.2
Optimizer	SGD, momentum 0.9
Weight decay	2×10^{-4}
Cosine warmup ratio	0.1
Reference training epochs	250

The central blocks use kernels 7, 11, and 15, each repeated three times, giving a wide temporal receptive field without self-attention. NeMo’s file notes that the historical original training used a higher LR than the current example; config-file LR must therefore not be assumed to reproduce the paper exactly.

7.2 TitaNet-S current reference configuration

TitaNet-S uses the same topology but reduces the main filters from 1024 to 256, keeping 192-D embeddings. NVIDIA describes the small model at roughly 6.4M parameters [9]. This demonstrates an important speaker-network design principle: the gallery embedding dimension can remain fixed while the frame encoder width is scaled independently.

7.3 Depthwise-separable parameter reduction

A standard 1D convolution with kernel K , input channels C_{in} , output channels C_{out} costs approximately

$$P_{full} = KC_{in}C_{out}. \quad (13)$$

Depthwise + pointwise costs

$$P_{sep} = KC_{in} + C_{in}C_{out}. \quad (14)$$

For large C_{out} , this is dramatically smaller, explaining why TitaNet can afford wide temporal kernels.

8 CAM++ / CAMPPlus

CAM++ targets ECAPA-like accuracy with TDNN-like efficiency by combining a frequency convolution module (FCM), densely connected TDNN blocks, context-aware masking, and multi-granularity/statistical pooling [10]. Current 3D-Speaker/FunASR implementation settings are especially useful for reproducing the network [11, 12].

8.1 Current CAMPPlus parameters

- input feature dimension: 80;
- common embedding size: 192 (VoxCeleb variant in 3D-Speaker may expose 512);
- FCM: initial 3×3 Conv2D, 32 base channels, residual block counts [2, 2];
- FCM frequency downsampling: two residual strides plus a Conv2D stride (2, 1), producing $32 \times (80/8) = 320$ flattened channels;
- initial TDNN: $320 \rightarrow 128$, kernel 5, stride 2;
- Dense-CAM block 1: 12 layers, kernel 3, dilation 1;
- Dense-CAM block 2: 24 layers, kernel 3, dilation 2;
- Dense-CAM block 3: 16 layers, kernel 3, dilation 2;
- growth rate 32;
- bottleneck factor $bn_size = 4$, so bottleneck channels are $4 \times 32 = 128$;
- transition layer after each dense block halves channels;
- final statistics pooling concatenates mean/std;
- final DenseLayer produces 192-D embedding in the common model.

Independent implementations report about 6.8M encoder parameters for a common CAM++ setup, excluding classifier parameters; this figure should be treated as implementation-specific rather than universal [13].

8.2 Why the dense growth rate matters

If a dense block begins with C_0 channels and has L layers with growth rate g , then before transition the output width grows approximately as

$$C_L = C_0 + Lg. \quad (15)$$

CAM++ then halves the width in a transition layer. A small $g = 32$ lets the model add many intermediate features without the large fixed width of ECAPA-1024.

9 MFA-Conformer

MFA-Conformer combines convolutional subsampling, Conformer blocks, multi-layer feature aggregation, attentive statistics pooling, and a final embedding layer [14].

9.1 Paper configuration

The original paper specifies:

- input: 80-D FBANK, 25-ms window, 10-ms hop;
- random 3-s training segments;
- Conformer encoder dimension $d = 256$;
- 4 self-attention heads, hence head dimension 64;
- convolution kernel size 15;
- FFN hidden dimension 2048 ($8 \times$ the encoder dimension);
- 6 Conformer blocks;
- multiple tested convolutional subsampling rates; $1/2$ is a strong practical setting;
- concatenate outputs from all six blocks before LayerNorm and attentive statistics pooling;
- final speaker embedding commonly configured to 192 dimensions in released/reference systems;
- approximately 19.7–20.5M learnable parameters depending on subsampling variant;
- AM-Softmax margin 0.2, scale 30;
- Adam, initial LR 0.001, LR halved every 4 epochs, warmup 2k steps;
- batch size 200; weight decay 10^{-7} [14].

9.2 Conformer block equations

A Macaron-style Conformer block can be summarized as

$$\tilde{\mathbf{h}} = \mathbf{h} + \frac{1}{2}\text{FFN}(\mathbf{h}), \quad (16)$$

$$\mathbf{h}' = \tilde{\mathbf{h}} + \text{MHSA}(\tilde{\mathbf{h}}), \quad (17)$$

$$\mathbf{h}'' = \mathbf{h}' + \text{Conv}(\mathbf{h}'), \quad (18)$$

$$\mathbf{y} = \text{LN} \left(\mathbf{h}'' + \frac{1}{2}\text{FFN}(\mathbf{h}'') \right). \quad (19)$$

The self-attention cost grows roughly with sequence length squared, making subsampling an architectural, not merely optimization, parameter.

10 Self-supervised Transformer encoders: WavLM and UniSpeech-SAT

Large self-supervised models can act as frozen or fine-tuned front-end encoders. Speaker identification then adds weighted layer selection, TDNN/attention pooling, and a speaker loss.

10.1 WavLM Base / Base+

Current Microsoft/Hugging Face metadata gives [16]:

- 7-layer waveform convolutional feature extractor;
- convolution channels all 512;
- kernels [10, 3, 3, 3, 3, 2, 2];
- strides [5, 2, 2, 2, 2, 2, 2];
- Transformer hidden size 768;
- 12 Transformer layers;
- 12 attention heads (64-D per head);
- FFN/intermediate size 3072;
- hidden dropout 0.1; layerdrop 0.05;
- 16-kHz raw-waveform input.

A released WavLM Base+ speaker-verification configuration adds an x-vector-style TDNN head with dimensions [512, 512, 512, 512, 1500], kernels [5, 3, 3, 1, 1], dilations [1, 2, 3, 1, 1], weighted layer sum, and a 512-D x-vector output [18].

10.2 WavLM Large

WavLM Large uses [17]:

- hidden size 1024;
- 24 Transformer layers;
- 16 attention heads;
- FFN 4096;
- same seven-stage convolutional waveform feature extractor pattern;
- substantially larger memory and compute than Base.

The WavLM paper scales pretraining to approximately 94k hours and combines masked prediction with denoising [15]. For SID, the practical choice is often to freeze early layers initially, learn a speaker pooling/head, then selectively unfreeze.

10.3 UniSpeech-SAT

UniSpeech-SAT is important structurally because it adds speaker-aware self-supervision to a HuBERT-like encoder: utterance-level contrast and utterance mixing encourage the representation to retain speaker identity [19]. When labeled speaker data are scarce, this pretraining objective can be more relevant than generic image or audio-event pretraining.

11 Audio Spectrogram Transformer and AST-Tiny for SID

AST treats the spectrogram as a sequence of 2D patches [20]. The original AST code supports a tiny configuration based on distilled DeiT-Tiny. The underlying DeiT-Tiny distilled architecture is precisely specified as patch size 16, embedding dimension 192, depth 12, 3 attention heads, MLP ratio 4, and LayerNorm epsilon 10^{-6} [21]. Current timm metadata also reports a 192-D feature output [22].

11.1 AST-Tiny speaker-identification structure

For 16-kHz speaker identification, a practical adaptation is:

1. 80 or 128 log-Mel bins;
2. fixed or padded time frames, e.g. 512 frames $\approx$ 5.12 s at 10-ms hop;
3. patch size 16×16 ;
4. frequency stride $s_f = 10$, time stride $s_t = 10$ for overlapping patches;
5. one-channel patch projection to 192 dimensions;
6. CLS + distillation tokens plus positional embedding;
7. 12 Transformer blocks, 3 heads, head dimension 64;
8. MLP hidden dimension $4 \times 192 = 768$;
9. pool (CLS + DIST)/2 or add speaker-specific attentive/statistics pooling;
10. speaker embedding projection 192 or 256;
11. AAM-Softmax training head over training speakers;
12. cosine/prototype scoring for dynamic enrollment.

For $F = 128$, $T = 512$, patch 16×16 , stride 10×10 ,

$$N_F = \left\lfloor \frac{128 - 16}{10} \right\rfloor + 1 = 12, \quad (20)$$

$$N_T = \left\lfloor \frac{512 - 16}{10} \right\rfloor + 1 = 50, \quad (21)$$

$$N_{patch} = 600. \quad (22)$$

Self-attention therefore operates on 602 tokens including CLS and DIST. Increasing T from 512 to 1024 nearly doubles token count and can roughly quadruple the dominant attention matrix work, so target duration is a critical AST parameter.

11.2 Pretraining interpretation

For AST-Tiny, the original AST code path uses ImageNet-pretrained distilled DeiT-Tiny when AudioSet pretraining is not available for the tiny model. This is valid transfer learning, but not audio-domain speaker pretraining. Consequently, an AST-Tiny SID experiment should be compared with a speaker-pretrained ECAPA/TitaNet/CAM++ model under identical data splits.

12 Pooling structures and parameters

Pooling is the boundary between frame/patch representation and speaker identity.

12.1 Mean and statistics pooling

Mean pooling uses $d = C$ output dimensions. Mean+standard-deviation pooling uses $2C$. It has no learned parameters and is highly stable.

12.2 Attentive statistics pooling

With attention bottleneck R and input C , a simple shared attention MLP has parameter order $O(CR + R)$; channel-dependent attention can increase this. Smaller R regularizes attention, while too-small R may underfit heterogeneous channels. ECAPA commonly uses $R = 128$.

12.3 Multi-layer aggregation

Concatenating L layers of width C creates LC channels before reduction. ECAPA concatenates hierarchical layers; MFA-Conformer concatenates all Conformer-block outputs. This exposes multiple abstraction depths to pooling but raises projection/pooling cost.

12.4 CLS-token pooling

Transformer models can use CLS token, but SID often benefits from pooling that explicitly aggregates distributional statistics over time. If AST is used for SID, CLS-only should be treated as a baseline, not automatically the final design.

13 Speaker embedding dimension

Common embedding dimensions include 192, 256, and 512.

For a gallery of N speakers using FP32, prototype memory is

$$M = 4Nd \text{ bytes.} \quad (23)$$

For $N = 1,000,000$:

- $d = 192$: about 768 MB;
- $d = 256$: about 1.024 GB;
- $d = 512$: about 2.048 GB.

FP16 halves these values. Cosine-search arithmetic scales linearly with d . Therefore 192-D embeddings are attractive for large galleries and are used by ECAPA, TitaNet, MFA-Conformer references, CAM++ common models, and ERes2NetV2 recipes.

Increasing d helps only if the encoder and data can use the extra capacity; otherwise it can increase overfitting and search cost without improving separation.

14 Loss-head parameters

14.1 Cross entropy

For fixed training speakers:

$$\mathcal{L}_{CE} = -\log \frac{e^{a_y}}{\sum_j e^{a_j}}. \quad (24)$$

It is a useful debugging baseline but does not explicitly enforce angular separation.

14.2 AAM-Softmax

Normalize embedding and class weights. For target class y ,

$$\mathcal{L}_{AAM} = -\log \frac{e^{s \cos(\theta_y + m)}}{e^{s \cos(\theta_y + m)} + \sum_{j \neq y} e^{s \cos \theta_j}}. \quad (25)$$

Two central parameters are:

Scale s	Often 30. It controls softmax logit magnitude after normalization.
Margin m	Often 0.2 in baseline training; large-margin fine-tuning may increase it to around 0.3–0.5 depending on recipe.

TitaNet’s current config uses scale 30 and margin 0.2 [8]; MFA-Conformer uses AM-Softmax scale 30, margin 0.2 [14].

14.3 Classifier parameter count

For embedding d and N training speakers, the classification matrix alone has roughly dN parameters. With $d = 192$ and $N = 100,000$, this is 19.2M parameters – possibly larger than the encoder. This classifier is normally discarded at deployment, so papers must state whether reported model parameters include the training head.

15 Training and optimization parameters

Table 4: High-impact training hyperparameters.

Parameter	Typical starting range	Effect
Random crop	2–4 s	Longer crops stabilize identity evidence; shorter crops increase batch diversity.
Batch sampling	multiple utterances per speaker or balanced speakers	Prevents high-resource speakers dominating; required for GE2E-like losses.
Batch size	64–256+	Affects class diversity and BatchNorm; large speaker batches often help margin losses.
Optimizer	AdamW / Adam / SGD	SGD is common in some speaker recipes (TitaNet); Adam is common in Conformer systems.
Encoder LR	10^{-5} – 10^{-3} for transfer; higher from scratch	Pretrained Transformer backbones usually need smaller LR than new heads.
Weight decay	10^{-7} – 10^{-3} model-dependent	Interacts strongly with normalization and optimizer.
AAM margin	0.2 baseline; larger in LMFT	Larger margin sharpens separation but can destabilize early training.
Augmentation prob	0.5–0.9 overall	Noise/RIR/channel augmentation is essential for nuisance invariance.
SpecAugment	modest masks	Helpful for spectrogram models; excessive masking can remove speaker cues.

16 Closed-set versus open-set network-head design

16.1 Closed-set classifier

For a permanently fixed roster N , the deployed head may remain $\text{Linear}(d, N)$. This gives direct posterior-like scores but cannot naturally enroll a new speaker without updating the head.

16.2 Open-set embedding system

Train with many development identities, discard the classifier, enroll target speakers by prototype, and tune unknown threshold τ on a separate development set. The network parameters and backend parameters are therefore distinct:

$$\Theta_{\text{system}} = \{\theta_{\text{encoder}}, d, \text{pool}, m, s, K_{\text{enroll}}, \tau, K_{\text{ANN}}\}. \quad (26)$$

K_{enroll} is number of enrollment clips; K_{ANN} is candidate count for large-gallery approximate search.

17 Architecture comparison

Table 5: Representative network structures. Values are tied to the stated reference, not universal model-name constants.

Family	Input	Core width/depth	Embedding	Key structural parameters
x-vector	24 FBANK original	512 TDNN, frame5 1500	512	contexts 5/9/15; stats pool 3000; 4.2M deployment encoder [1]
ECAPA large	80 FBANK typical	$C = 1024$, 3 SE-Res2	192	dilations 2/3/4, scale 8, SE/attn bottleneck 128, MFA 1536 [2]
ERes2NetV2	80 FBANK	$m_channels = 64$	192	baseWidth 26, scale 2, expansion 2, 17.8M [6]
TitaNet-L	80 Mel	1024, repeats 3	192	DW-separable kernels 3/7/11/15, SE, final 3072 [8]
TitaNet-S	80 Mel	256, repeats 3	192	same topology, roughly 6.4M [9]
CAM++ common	80 FBANK	growth 32, init 128	192	FCM 32ch, dense blocks 12/24/16, CAM, transitions half width [11, 12]
MFA-Conformer	80 FBANK	6 blocks, $d = 256$	192 ref.	4 heads, FFN 2048, conv k15, MFA, 20M [14]
WavLM Base+	raw 16 kHz	12×768	512 SV head	12 heads, FFN3072, 7-layer conv frontend, TDNN head [16, 18]
WavLM Large	raw 16 kHz	24×1024	task head	16 heads, FFN4096, large SSL backbone [17]
AST-Tiny SID	80/128 log-Mel	12×192	192/256	patch16, 3 heads, MLP768, overlapping audio patches [21, 20]

18 Parameter-sensitivity analysis

18.1 Channels versus depth

Increasing channel width increases representation capacity in every frame, but convolutional parameter count typically grows approximately quadratically with width. Increasing depth expands hierarchical context and nonlinearity, often with lower per-layer parameter increase. Small datasets usually benefit more from pretraining and augmentation than simply doubling width.

18.2 Kernel size versus dilation

For a 1D convolution with kernel K and dilation d , effective receptive field is

$$R = 1 + (K - 1)d. \quad (27)$$

Stacked dilated blocks rapidly enlarge context. ECAPA uses $K = 3$ with increasing dilations; TitaNet instead uses large kernels 7, 11, 15 with depthwise separable convolution. These are two different efficiency strategies for long temporal context.

18.3 Attention heads

With model dimension D and H heads, head dimension is $d_h = D/H$. Too many heads at small D produce tiny subspaces; too few heads reduce diversity. MFA-Conformer uses $D = 256, H = 4 \Rightarrow d_h = 64$. DeiT/AST-Tiny uses $D = 192, H = 3 \Rightarrow d_h = 64$. WavLM Large uses $D = 1024, H = 16 \Rightarrow d_h = 64$. The repeated 64-D head dimension is a useful design pattern, not a hard rule.

18.4 Embedding dimension versus classifier size

Increasing embedding d increases both gallery cost and training classifier parameters dN . For very large N , a modest $d = 192$ is computationally attractive. If using 100k training speakers, changing 192 to 512 adds 32M classifier weights.

19 Transfer learning parameter strategy

19.1 General two-stage recipe

For a pretrained speaker or audio encoder:

1. replace task/classification head;
2. freeze backbone for 1–5 epochs and train new head;
3. unfreeze all or top blocks;
4. set backbone LR 5–20 $\times$ smaller than head LR;
5. continue with augmentation and AAM loss;
6. optionally run large-margin fine-tuning with longer crops and lower LR.

19.2 AST-Tiny-specific starting values for 16-kHz SID

Table 6: Recommended AST-Tiny transfer-learning baseline for 16-kHz speaker identification.

Parameter	Starting value
Sample rate	16,000 Hz
Mel bins	128 (also benchmark 80)
Window / hop	25 ms / 10 ms
Target frames	512 (≈ 5.12 s)
Patch / stride	16×16 / 10×10
Backbone	distilled DeiT-Tiny
Transformer depth	12
Embedding width	192
Heads	3
MLP hidden	768
Speaker embedding	192
Loss	AAM-Softmax
AAM s, m	30, 0.2
Head warmup LR	10^{-3}
Backbone fine-tune LR	10^{-5}
Head fine-tune LR	10^{-4}
Head-only epochs	3
Full fine-tune	15–30 epochs + early stop
Freq/time masks	modest; e.g. 8–12 Mel bins / 20–40 frames
Scoring	cosine on L2-normalized embeddings

This should be compared against an ECAPA-192 baseline trained/fine-tuned on the same split. If AST only beats a simple classifier but not ECAPA, the improvement is not architecture-competitive for SID.

20 Three engineering reference configurations

20.1 Edge / low-compute system

Recommended: TitaNet-S or CAM++.

- 16 kHz, 80 Mel;
- 2–3 s crop;
- 192-D embedding;
- AAM $s = 30, m = 0.2$;
- INT8/FP16 benchmark after training;
- cosine search; ANN only for very large galleries.

Priority metrics: EER/DIR, real-time factor, peak RAM, model size.

20.2 Balanced general-purpose system

Recommended: ECAPA-TDNN $C = 512$ or 1024 depending hardware.

- 80-D FBANK;
- 3 SE-Res2 blocks, scale 8, dilations 2/3/4;
- attentive stats + 192-D embedding;
- AAM, augmentation, multi-utterance enrollment;
- open-set threshold calibrated at deployment gallery scale.

20.3 High-accuracy server system

Recommended candidates: ERes2NetV2 / deep ResNet / MFA-Conformer / WavLM Large + speaker head.

- large diverse pretraining corpus;
- 3–6 s crops, large batch;
- large-margin fine-tune;
- score normalization only if validated;
- multi-utterance enrollment and top- K reranking.

Model selection must include latency, not only EER.

21 Implementation checklist

Freeze every item below in configuration management:

1. sample rate, channel conversion, resampler;
2. VAD behavior and minimum active speech duration;
3. Mel/FFT/window/hop/normalization;
4. crop length and padding/cropping rule;
5. exact encoder class and channel/layer parameters;
6. pooling class and bottleneck dimensions;
7. embedding dimension and normalization point;
8. loss type, margin, scale, number of training classes;
9. optimizer, LR schedule, warmup, weight decay, grad clipping;
10. augmentation probabilities and SNR/RIR distributions;
11. enrollment utterance count and prototype rule;
12. cosine/PLDA scoring and any score normalization;
13. open-set threshold and the target FAR used to select it;
14. model version hash, feature version, and template compatibility.

22 Conclusions

Speaker-identification model structure is best understood as a sequence of design choices rather than a single architecture label. The critical structural parameters are temporal/frequency representation, frame-encoder width and receptive field, multi-scale mechanism, attention/pooling, embedding dimension, and training loss geometry. Modern speaker-specific systems converge on several practical values: 16-kHz audio, roughly 80 Mel bins, 2–4 s training crops, 192-D embeddings, and angular-margin training. They differ mainly in how they construct temporal context: x-vector uses spliced TDNN context, ECAPA uses dilated SE-Res2 blocks, TitaNet uses large depthwise-separable kernels, CAM++ uses dense TDNN growth with context-aware masks, Res2Net families use internal multi-scale 2D convolution, MFA-Conformer combines local convolution with global self-attention, and WavLM uses large-scale self-supervised Transformers.

For a new speaker-identification project, the highest-confidence implementation path is to establish an ECAPA-TDNN or CAM++/TitaNet baseline before introducing a generic Transformer. AST-Tiny is technically suitable for transfer learning, especially when a pretrained DeiT/AST stack already exists, but speaker-specific pooling and AAM training are necessary, and its value should be demonstrated against a dedicated speaker encoder under identical data and open-set evaluation conditions.

A Appendix A: concise formulas for parameter budgets

A.1 1D convolution

$$P = KC_{in}C_{out} + C_{out} \quad (\text{with bias}).$$

A.2 Depthwise-separable 1D convolution

$$P = KC_{in} + C_{in}C_{out}.$$

A.3 Transformer self-attention projections

Ignoring biases, Q/K/V/output projections are approximately

$$P_{attn} \approx 4D^2.$$

A two-layer MLP with ratio r is approximately

$$P_{mlp} \approx 2rD^2.$$

For DeiT-Tiny $D = 192, r = 4$, a block’s dense weights are on the order of

$$(4 + 8)D^2 = 12 \cdot 192^2 \approx 442,000$$

before norms/biases, consistent with a compact 12-block backbone.

B Appendix B: recommended open-set backend parameters

Parameter	Starting recommendation
Enrollment clips	3–5 per speaker when possible
Prototype	L2 normalize each; average; L2 normalize again
Probe	one L2-normalized embedding
Similarity	cosine
Candidate search	exact if gallery small; ANN top-20 for large gallery
Threshold	select on disjoint unknown speakers
Metric	DIR@FAR plus Top-1 for known probes
Threshold revalidation	whenever gallery size/domain/model changes

References

- [1] D. Snyder, D. Garcia-Romero, G. Sell, D. Povey, S. Khudanpur, “X-Vectors: Robust DNN Embeddings for Speaker Recognition,” ICASSP 2018. https://danielpovey.com/files/2018_icassp_xvectors.pdf
- [2] B. Desplanques, J. Thienpondt, K. Demuynck, “ECAPA-TDNN: Emphasized Channel Attention, Propagation and Aggregation in TDNN Based Speaker Verification,” Interspeech 2020. <https://arxiv.org/abs/2005.07143>
- [3] Y. Lei et al., “The 2022 Far-field Speaker Verification Challenge,” 2022. https://www.isca-archive.org/ffsvc_2022/lei22_ffsvc.pdf
- [4] Y. Chen et al., “An Enhanced Res2Net with Local and Global Feature Fusion for Speaker Verification,” Interspeech 2023. <https://arxiv.org/abs/2305.12838>
- [5] 3D-Speaker, ERes2NetV2 VoxCeleb training configuration. <https://github.com/modelscope/3D-Speaker/blob/main/egs/voxceleb/sv-eres2netv2/conf/eres2netv2.yaml>
- [6] 3D-Speaker, ERes2NetV2 recipe and results. <https://github.com/modelscope/3D-Speaker/blob/main/egs/voxceleb/sv-eres2netv2/README.md>
- [7] N. R. Koluguri, T. Park, B. Ginsburg, “TitaNet: Neural Model for Speaker Representation with 1D Depth-Wise Separable Convolutions and Global Context,” ICASSP 2022. <https://arxiv.org/abs/2110.04410>
- [8] NVIDIA NeMo, TitaNet-L and TitaNet-S speaker-recognition configurations. https://github.com/NVIDIA/NeMo/Speech/blob/main/examples/speaker_tasks/recognition/conf/titanet-large.yaml
- [9] NVIDIA NGC, TitaNet-S model card. https://catalog.ngc.nvidia.com/orgs/nvidia/nemo/models/titanet_small
- [10] H. Wang, S. Zheng, Y. Chen, L. Cheng, Q. Chen, “CAM++: A Fast and Efficient Network for Speaker Verification Using Context-Aware Masking,” Interspeech 2023. <https://arxiv.org/abs/2303.00332>
- [11] 3D-Speaker, CAMPPlus reference implementation. <https://github.com/modelscope/3D-Speaker/blob/main/speakerlab/models/campplus/DTDNN.py>
- [12] FunASR, CAMPPlus config. <https://huggingface.co/funasr/campplus/blob/main/config.yaml>
- [13] VoiceprintRecognition-Pytorch comparative model parameter table (implementation-specific). https://gitee.com/dymayun/VoiceprintRecognition-Pytorch/blob/develop/README_en.md
- [14] Y. Zhang et al., “MFA-Conformer: Multi-scale Feature Aggregation Conformer for Automatic Speaker Verification,” Interspeech 2022. <https://arxiv.org/abs/2203.15249>
- [15] S. Chen et al., “WavLM: Large-Scale Self-Supervised Pre-Training for Full Stack Speech Processing,” 2021/2022. <https://arxiv.org/abs/2110.13900>
- [16] Microsoft, WavLM Base/Base+ configuration metadata. <https://huggingface.co/microsoft/wavlm-base-plus/blob/main/config.json>
- [17] Microsoft, WavLM Large configuration metadata. <https://huggingface.co/microsoft/wavlm-large/blob/main/config.json>

- [18] Microsoft, WavLM Base+ speaker-verification x-vector head configuration. <https://huggingface.co/microsoft/wavlm-base-plus-sv/blob/main/config.json>
- [19] S. Chen et al., “UniSpeech-SAT: Universal Speech Representation Learning with Speaker Aware Pre-Training,” ICASSP 2022. <https://arxiv.org/abs/2110.05752>
- [20] Y. Gong, Y.-A. Chung, J. Glass, “AST: Audio Spectrogram Transformer,” Interspeech 2021. <https://arxiv.org/abs/2104.01778>
- [21] Facebook Research, DeiT model definitions, distilled DeiT-Tiny: patch 16, embed 192, depth 12, 3 heads, MLP ratio 4. <https://github.com/facebookresearch/deit/blob/main/models.py>
- [22] timm, deit_tiny_distilled_patch16_224.fb_in1k metadata. https://huggingface.co/timm/deit_tiny_distilled_patch16_224.fb_in1k/blob/main/config.json